

Aufgabe 3: Eisbudendilemma

Team-ID: 00048

Team: `while(!success){try();}`

Bearbeiter/-innen dieser Aufgabe:
Rui Zhang

25. December 2020

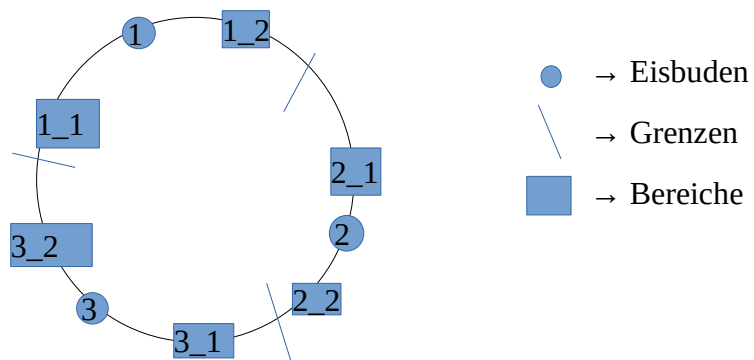
Inhaltsverzeichnis

Lösungsidee.....	1
Umsetzung.....	3
Beispiele.....	5
Quellcode.....	7

Lösungsidee

Die Aufgabenstellung der 3. Aufgabe ist es, stabile Eisbudenstandorte zu finden. Das heißt, dass nach dem Festlegen der Eisbudenstandorte eine Verlegung nicht mehr zustande kommen kann, da es immer dazu kommen wird, dass es mehr Stimmen gegen die Verlegung als Stimmen für die Verlegung gibt. Das bedeutet also, dass das Ziel daraus besteht, die Eisbuden so zu positionieren, dass sie sagesagt auf “beiden Seiten” genau so viele Stimmen besitzen.

Da das Stimmen für eine Verlegung einer Eisbude nur die Häuser betrifft, die am nächsten zu dieser Eisbude liegen, müssen Grenzen berechnet werden. Zwischen diesen Grenzen und den einzelnen Positionen der Eisbuden liegen Häuser, die im Falle einer Verlegung der nächstliegenden Bude in die entgegengesetzte Richtung dagegen stimmen werden.



Es ergeben sich hiermit Grenzen, deren Positionen mithilfe der Positionen der Eisbuden zu berechnen sind und damit auch die Bereiche 1_1, 1_2, 2_1, 2_2, 3_1 und 3_2. Im Falle einer Verschiebung stimmen alle Bewohner im Bereich 1_1 gegen die Verschiebung, falls die Verschiebung in Richtung 1_2 ist und für die Verschiebung, falls die Verschiebung in Richtung 1_1 ist. Dies gilt auch für alle anderen Bereiche. Wenn also die Positionen der Eisbuden 1, 2 und 3 so gewählt sind, dass die Anzahl an Häusern im Bereich 1_1 gleich im Bereich 1_2 ist, die Anzahl im Bereich 2_1 gleich im Bereich 2_2 und die Anzahl im Bereich 3_1 gleich im Bereich 3_2 ist, dann wurden stabile Positionen gefunden. Die Anzahl der Häuser in den Bereichen werden als "Mengen" bezeichnet:

Zusammenfassung:

Stabile Positionen falls:

$\text{Menge1_1} == \text{Menge1_2}$,

$\text{Menge2_1} == \text{Menge2_2}$ und

$\text{Menge3_1} == \text{Menge3_2}$

Hierbei kann es jedoch dazu kommen, dass Häuser an der selben Adresse wie eine Eisbude stehen. In diesem Fall stimmen diese Häuser gegen überhaupt irgendeine Verschiebung der Eisbude, da diese zu einer Verlängerung des Weges zur nächstliegenden Bude führt. Das bedeutet, dass auch ein Unterschied zwischen zweier Mengen von einem Haus akzeptabel ist, da bei jeder Verschiebung das Haus, das mal an der selben Stelle wie die Eisbude stand, gegen die Verschiebung stimmen würde.

Beispiel:

Menge1_1: 2,

Menge1_2: 3,

Haus an selber Stelle wie Eisbude 1,

Verschiebung in Richtung Menge1_1: → 4 stimmen dagegen, 2 stimmen dafür → keine Verschiebung

Verschiebung in Richtung Menge1_2: → 3 stimmen dagegen, 3 stimmen dafür → auch keine Verschiebung, da es keine Mehrheit gibt

Das bedeutet also, dass die Bedingung:

$\text{Menge1_1} == \text{Menge1_2}$ oder $(\text{absoluterWert}(\text{Menge1_1} - \text{Menge1_2}) == 1$ und Haus an stelle von Eisbude 1),

$\text{Menge2_1} == \text{Menge2_2}$ oder $(\text{absoluterWert}(\text{Menge2_1} - \text{Menge2_2}) == 1$ und Haus an stelle von Eisbude 2) und

$\text{Menge3_1} == \text{Menge3_2}$ oder $(\text{absoluterWert}(\text{Menge3_1} - \text{Menge3_2}) == 1$ und Haus an stelle von Eisbude 3)

die endgültige Bedingung für stabile Eisbudenpositionen darstellt. Somit muss ein Algorithmus implementiert werden, der alle möglichen Kombinationen dreier Eisbudenpositionen ausprobiert und

1. die Positionen der Grenzen ausrechnet
2. die Mengen mithilfe der Adressen der Häuser bestimmt
3. die Mengen miteinander abgleicht und
4. die Positionen der Eisbuden ausgibt falls die Mengen von der Größe her übereinstimmen (s. Bedingung oben)

Falls jedoch alle Kombinationen ausprobiert wurden, so muss ausgegeben werden, dass es keine stabilen Eisbudenpositionen gibt.

Umsetzung

In dem eigentlichen Programm gibt der Nutzer zunächst eine Zahl n ein, woraufhin die n -te Beispiel-Datei von der BWINF-Webseite genutzt wird. Die Ausgabe wird sowohl in der Konsole ausgegeben als auch in die n -te Ausgabe-Datei aufgeschrieben.

Die nötigen Informationen für den Algorithmus wird von der Beispiel-Datei entnommen: See Umfang und die Adressen der Häuser.

Eine Funktion “finde_stabile_eisdielen_positionen” nimmt eine Liste an Adressen und den See Umfang und gibt entweder drei Positionen zurück, die die stabile Positionen darstellen, oder überhaupt nichts (None), falls es keine stabile Positionen gibt.

In der Funktion werden drei For-loops genutzt die drei Iteratoren position1, position2 und position3 besitzen. Sie sind so eingestellt, dass die Positionen im Kreis immer in der Reihenfolgen Eisdiele 1, 2 und dann erst 3 kommt. Das heißt, dass es nicht sein kann, dass zum Beispiel position3 an der Stelle 0, position2 an der Stelle 3 und position1 an der Stelle 6 vorkommen. Die Positionen sind immer in der Reihenfolge 1, 2, 3. Das wird durch den Intervall der For-loops versichert, indem position1 nur von 0 bis See_umfang – 2 geht, position2 von position1 + 1 bis See_umfang -1 und position3 nur von position2 bis See_umfang.

In dem innersten For-loop werden zunächst die Grenzen berechnet. Dies geht anhand dreier simplen Gleichungen:

$\text{grenze_1_2} = (\text{position2} - \text{position1}) / 2 + \text{position1}$ → Berechnung des Mittelpunktes zwischen p1 und p2

$\text{grenze_2_3} = (\text{position3} - \text{position2}) / 2 + \text{position2}$ → Berechnung des Mittelpunktes zwischen p2 und p3

$\text{grenze_3_1} = \text{position1} - (\text{see_umfang} - \text{position3} + \text{position1}) / 2$

falls $\text{grenze_3_1} < 0$ dann füge See_umfang zu grenze_3_1 hinzu

Die dritte Grenze wird anders berechnet als sonst, da die Positionen zwischen position1 und position3 nicht nur Zahlen in einer Reihenfolge sind. Hierbei ergibt sich die absolute Distanz zwischen der Positionen durch $\text{see_umfang} - \text{position3} + \text{position1}$. Position1 zieht dann die Hälfte der absoluten Distanz zwischen 3 und 1, woraus sich die Position der Grenze ergibt. Es kann jedoch sein, dass die Grenze jetzt negativ ist, woraufhin See_umfang hinzugefügt wird, sodass die Position eine positive Zahl ist.

Anschließend werden Variablen definiert, die die Größe der Mengen darstellen und wiedergeben, ob an der Stelle einer Eisbude auch ein Haus vorhanden ist.

Eine weitere For-Schleife geht durch alle Adressen der Häuser durch und fügt zu den entsprechenden Mengen eins hinzu, falls sie zu denen gehören. Bei den meisten Mengen funktioniert es recht simpel mit einfachen Vergleichen zwischen den Größen der Positionen der Eisdielen und der Positionen der Grenzen, schließlich muss die Adresse nur größer als die Position der Grenze und kleiner als die Position der Eisdielen sein, um zu einer bestimmten Menge zu gehören.

Bei der Menge3_2 und Menge1_1 ist der Prozess jedoch schwieriger, da auch dort die Zahlen nicht nur in einer Reihenfolge sind. Hierfür wird zunächst berechnet, auf welchem Halbkreis die Grenze 3_1 liegt.

Wenn die Position der Grenze kleiner ist als die Hälfte des See_umfangs → Grenze ist auf der rechten Hälfte des Kreises (wenn die Position 0 ganz nördlich positioniert ist und die Zahlen im Uhrzeigersinn verfolgen), so muss:

- zu der Menge1_1 hinzugefügt werden, wenn die Adresse einfach zwischen position1 und der Grenze liegen

- zu der Menge3_2 hinzugefügt werden, wenn die Adresse größer ist als position3 oder größer / gleich 0, dafür aber kleiner als die Position der Grenze

Wenn die Position der Grenze größer ist als die Hälfte des See_umfangs → Grenze ist auf der linken Hälfte des Kreises (wenn die Position 0 ganz nördlich positioniert ist und die Zahlen im Uhrzeigersinn verfolgen), so muss:

- zu der Menge_3_2 hinzugefügt werden, wenn die Adresse einfach zwischen position3 und der Grenze liegen
- zu der Menge_1_1 hinzugefügt werden, wenn die Adresse größer ist als die Position der Grenze oder größer / gleich 0, dafür aber kleiner als position1

Außerdem werden die Variablen, die aussagen, ob ein Haus an der Stelle einer Eisbude vorhanden ist, auf True bzw. Wahr gesetzt, falls die Adressen übereinstimmen.

Zuletzt wird gemäß der anfangs genannten Bedingung abgefragt, ob die Positionen stabil sind und falls ja zurückgegeben.

Außerhalb der drei For-Schleifen wird None zurückgegeben, da diese Stelle nur erreicht wird, falls alle Kombinationen der drei Positionen ausprobiert wurden und keine Kombination die Bedingung erfüllt hat.

Beispiele

Beispiel1:

20 7

0 2 3 8 12 14 15

Die stabile Standorten für die Eisdielen sind: (0, 2, 12)

Beispiel2:

50 15

3 6 7 9 24 27 36 37 38 39 40 45 46 48 49

Die stabile Standorten für die Eisdielen sind: (0, 24, 38)

Beispiel3:

50 16

2 7 9 12 13 15 17 23 24 35 38 42 44 45 48 49

Die stabile Standorten für die Eisdielen sind: (0, 15, 40)

Beispiel4:

100 19

6 12 23 25 26 28 31 34 36 40 41 52 66 67 71 75 80 91 92

Die stabile Standorten für die Eisdielen sind: (0, 31, 67)

Beispiel5:

247 24

2 5 37 43 72 74 83 87 93 97 101 110 121 124 126 136 150 161 185 200 201 230 234 241

Die stabile Standorten für die Eisdielen sind: (0, 95, 154)

Beispiel6:

437 36

4 12 17 23 58 61 67 76 93 103 145 154 166 170 192 194 209 213 221 225 239 250 281 299 312
323 337 353 383 385 388 395 405 407 412 429

Die stabile Standorten für die Eisdielen sind: (0, 170, 299)

Beispiel7:

625 40

12 15 27 30 32 110 114 117 121 123 132 150 184 210 240 241 262 268 271 285 289 292 297 300
302 310 330 364 384 402 408 409 416 420 425 430 431 437 528 550

Die stabile Standorten für die Eisdielen sind: (15, 123, 310)

Quellcode

Da die Funktion “finde_stabile_eisdielen_positionen” der Teil ist, der für das Programm entscheidend ist, wird nur der Code von dieser Funktion hier aufgeschrieben.

```
for position1 in range(see_umfang - 2):
```

```
    for position2 in range(position1 + 1, see_umfang - 1):
```

```
        for position3 in range(position2 + 1, see_umfang):
```

```
            # Berechnung der Grenzen:
```

```
            grenze_1_2 = (position2 - position1) / 2 + position1
```

```
            grenze_2_3 = (position3 - position2) / 2 + position2
```

```
            grenze_3_1 = position1 - (see_umfang - position3 + position1) / 2
```

```
            if grenze_3_1 < 0:
```

```
                grenze_3_1 += see_umfang
```

```
            # Festlegen der Mengen:
```

```
            menge_1_1 = 0
```

```
            menge_1_2 = 0
```

```
            position1_besetzt = False # -> gibt an, ob Eisdeile 1 an der selben Adresse wie ein Haus
```

ist

```
            menge_2_1 = 0
```

```
            menge_2_2 = 0
```

```
            position2_besetzt = False # -> gibt an, ob Eisdeile 2 an der selben Adresse wie ein Haus
```

ist

```
            menge_3_1 = 0
```

```
            menge_3_2 = 0
```

position3_besetzt = False # -> gibt an, ob Eiseile 3 an der selben Adresse wie ein Haus
ist

```
for adresse in haus_adressen:
```

```
    if adresse == position1:
```

```
        position1_besetzt = True
```

```
    if adresse == position2:
```

```
        position2_besetzt = True
```

```
    if adresse == position3:
```

```
        position3_besetzt = True
```

```
if position1 < adresse < grenze_1_2:
```

```
    menge_1_2 += 1
```

```
if grenze_1_2 < adresse < position2:
```

```
    menge_2_1 += 1
```

```
if position2 < adresse < grenze_2_3:
```

```
    menge_2_2 += 1
```

```
if grenze_2_3 < adresse < position3:
```

```
    menge_3_1 += 1
```

```
# "special case" für grenze_3_1 und die dazugehörigen Mengen:
```

```
if grenze_3_1 > (see_umfang / 2):
```

```
    if grenze_3_1 > adresse > position3:
```

```
        menge_3_2 += 1
```

```
    if adresse > grenze_3_1 or position1 > adresse >= 0:
```

```
        menge_1_1 += 1
```

```
elif grenze_3_1 < (see_umfang / 2):
```



```
if adresse > position3 or grenze_3_1 > adresse >= 0:
```

```
    menge_3_2 += 1
```

```
if position1 > adresse > grenze_3_1:
```

```
    menge_1_1 += 1
```

```
    if (menge_1_1 == menge_1_2 or (abs(menge_1_1 - menge_1_2) == 1 and  
position1_besetzt)) and \
```

```
        (menge_2_1 == menge_2_2 or (abs(menge_2_1 - menge_2_2) == 1 and  
position2_besetzt)) and \
```

```
            (menge_3_1 == menge_3_2 or (abs(menge_3_1 - menge_3_2) == 1 and  
position3_besetzt)):
```

```
        return position1, position2, position3
```

```
return None
```